

Sushant's Answers to the Take-Home Questionnaire

GreyOrange - Abacus Engineering Exercise

Candidate	Sushant
Submission date	August 2, 2026
Scope	Engineering Exercise only; Product Manager Exercise excluded
Status	SUBMISSION CANDIDATE - HOSTED URL PENDING

Run final release verification, deploy the tagged commit, and add the hosted URL before recruiter submission.

Response coverage

1. Brand and Store Onboarding
2. Product Master Ingestion
3. Real-Time RFID Inventory Feed
4. Identity, Access, and User Management
5. Replenishment Policy Ingestion and Execution

Executive Summary

I designed and implemented a multi-tenant backend that demonstrates one complete path:

Tenant/Store -> Zones/Devices -> Catalog/EPCs -> RFID -> Item State -> Inventory -> Policy -> Replenishment Task

The submitted scope uses Python 3.12, FastAPI, SQLAlchemy 2, Alembic, PostgreSQL 17, JWT authentication, Pytest, Docker Compose, and generated OpenAPI/Swagger. It has three deployable processes sharing one codebase and image: the API, a catalog worker, and an RFID/inventory event worker.

PostgreSQL is both the system of record and the durable work inbox. Kafka/Redpanda and S3/MinIO are deliberate production extensions, not executable dependencies in this take-home. This keeps the reviewer path small and reliable while preserving stable event, transition, and projection contracts that can later sit behind a broker.

The runnable demo creates two Orange stores so the full workflow stays easy to inspect. The automated suite separately exercises the assignment-sized 100-store onboarding case, including 200 representative readers.

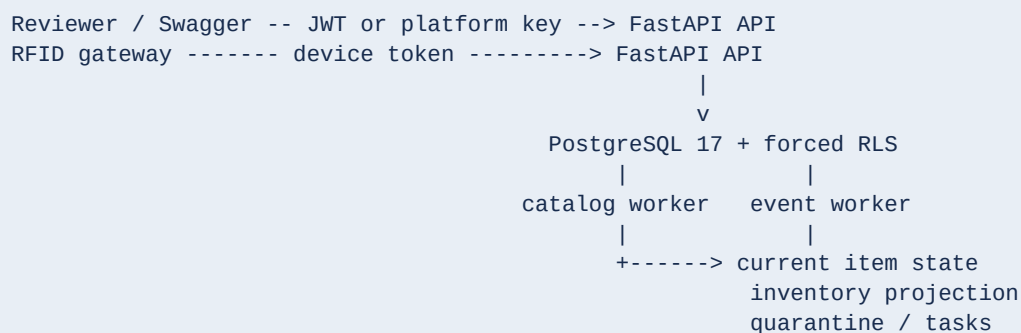
Submitted Demo Scope

The reviewer can:

- create Orange, stores, sales-floor/backroom zones, and readers;
- import a CSV product hierarchy and EPC mappings through a staged asynchronous job;
- submit device-authenticated RFID batches and poll their status;
- observe stable per-item locations and floor/backroom inventory;
- prove duplicate and late-event protection;
- create users and prove store-level authorization;
- create and activate a versioned policy; and
- evaluate replenishment and complete a deduplicated task.

The demo does not claim a frontend, real RFID hardware, Kafka, object storage, Kubernetes, regional cells, enterprise SSO/SCIM, full X.509 device lifecycle, precise XY positioning, or book-inventory variance.

Submitted Architecture



The API, catalog worker, and event worker can be deployed independently. The catalog worker promotes validated staging rows. The event worker drains the durable RFID inbox, resolves stable item state, and applies deduplicated inventory deltas. A worker outage leaves committed work in PostgreSQL for retry.

1. Brand and Store Onboarding

Tenant and Multi-Tenancy Model

Orange is one Abacus tenant. I did not add a separate **brand** entity because the exercise uses “brand/tenant” as the customer boundary and does not describe one customer owning several brands. If that requirement appears later, **brand** can be added beneath the tenant without changing the isolation boundary.

I chose a shared PostgreSQL schema with **tenant_id** on tenant-owned rows.

Choice	Benefit	Cost and mitigation
Shared schema	Fast provisioning, simple joins and migrations, economical for 100 stores	A bad query could cross tenants; forced PostgreSQL RLS is the backstop
Schema per tenant	More namespace separation	Migration and connection-pool overhead grows with customer count
Database per tenant	Strongest physical isolation	Highest cost and operational complexity; reserve for contractual or regulatory needs

Runtime connections use a **NOSUPERUSER, NOBYPASSRLS** application role. At the start of each tenant transaction the service sets **SET LOCAL app.tenant_id**; forced RLS policies compare each row with that setting and fail closed if it is absent. Only Alembic uses the owner credential; application sessions always use the restricted URL. Tenant identity comes from the authenticated principal or trusted platform credential, never from a request-body field.

Store Hierarchy and Configuration

The model is **Tenant -> Organization Unit -> Store -> Zone -> Device Assignment**. Stable IDs and fields used in joins are relational. A store has a tenant-qualified code, name, IANA timezone, lifecycle status, optional organization path, and sparse JSONB for genuinely variable customer metadata.

Orange can keep its own zone code and display name while the demo maps the zone to a normalized semantic kind such as **SALES_FLOOR** or **BACKROOM**. In a broader product, I would replace the fixed kind vocabulary with customer-defined zones mapped to platform roles such as available sales floor and replenishment source. That separates Orange's naming from inventory semantics without weakening the demo contract.

Core attributes such as timezone do not belong in a generic configuration registry. Tunable behavior belongs in typed, validated, versioned settings. Unknown presentation-only metadata can remain sparse JSONB.

Hardware-to-Store Mapping

A device is tenant-owned and has effective-dated assignments to a store and zone. The device asserts only its identity; the server resolves tenant, store, and zone from the assignment valid at **observed_at**. This prevents a compromised or misconfigured reader from choosing its own security scope.

The implementation uses a PostgreSQL GiST exclusion constraint over the tenant, device, and effective-time range, so neither active nor historical assignments can overlap.

The demo returns a device credential once and stores only its hash. Production hardware onboarding would add mTLS/X.509 rotation, inventory reconciliation, and model-specific antenna configuration after Orange supplies the gateway contract.

Provisioning Workflow

1. A trusted platform operator creates Orange.
2. A deterministic store-import request contains store metadata, organization paths, zones, and optional readers.
3. The API validates the entire request before writing.
4. The client supplies an **Idempotency-Key** and submits the batch.
5. Metadata is accepted atomically; an exact retry returns the same result and a conflicting reuse returns **409**.
6. In production, each store then advances independently through credentials, heartbeat, role coverage, walk test, and catalog readiness gates.
7. A readiness projection shows which gate blocks each store without putting transient commissioning state on the store row.

Atomic metadata acceptance prevents an internally inconsistent estate. Independent commissioning prevents one failed physical installation from blocking the other 99 stores. At 5,000 stores the platform workflow remains resumable, but walk tests must become technician-driven and self-service because physical commissioning, not database throughput, is the human scaling constraint.

Implemented APIs

- **POST /v1/tenants**
- **POST /v1/tenants/{tenant_id}/store-imports**
- **GET /v1/tenants/{tenant_id}/stores**
- **POST /v1/stores/{store_id}/zones**
- **GET /v1/stores/{store_id}/zones**
- **POST /v1/stores/{store_id}/devices**

2. Product Master Ingestion

Data Model and Pipeline

The canonical hierarchy is **Product -> Product Variant -> SKU -> RFID Tag**. An EPC identifies one physical item and maps to one SKU. Tenant-qualified uniqueness protects SKU, UPC mapping, and (**tenant_id, epc**).

The hosted flow is:

```
CSV upload -> checksum/idempotency check -> staging rows -> validation -> atomic promotion
```

The API records the source checksum and creates an asynchronous import. Parsed rows remain in PostgreSQL staging tables until every row passes. The catalog worker promotes the batch in one transaction, so readers never see a half-updated catalog. An exact re-import is idempotent; conflicting reuse of an idempotency key is rejected.

Validation covers required columns, formats, duplicate EPCs, conflicting SKU/UPC mappings, Product-to-Variant-to-SKU-to-EPC consistency, and required color/size data. Category is optional unless Orange enables category-scoped policies. Errors retain row and field context and are available through the errors endpoint.

Tradeoff

PostgreSQL staging is the best take-home choice because it gives constraints, atomic promotion, and one dependency. Its downside is that large source files consume database storage and worker time. At sustained enterprise feed sizes I would upload the original file to S3-compatible storage, scan it, parse in chunks, and bulk-load staging with **COPY**; the promotion transaction and public API can remain unchanged.

Orange still needs to confirm whether an omitted row in a full feed means deactivation, how corrections are versioned, and whether product, UPC, or EPC is authoritative. Those are business contracts, not safe engineering assumptions.

Implemented APIs

- **POST /v1/tenants/{tenant_id}/catalog-imports**
- **GET /v1/catalog-imports/{import_id}**
- **GET /v1/catalog-imports/{import_id}/errors**

3. Real-Time RFID Inventory

Ingestion and Durability

A gateway sends a device-authenticated observation batch. The API resolves its current assignment, creates the batch, creates or links each durable (**tenant_id, event_id**) ledger row, and inserts its inbox record in one PostgreSQL transaction. Only then does it return **202 Accepted** with a **batch_id**.

This is at-least-once processing with idempotent effects:

- the same event ID and same payload is a retry, not new evidence;
- the same event ID with a different payload is a conflict;
- a late event is retained but cannot regress newer current state;
- unknown EPCs and invalid-assignment evidence are quarantined; malformed requests

are rejected before durable acceptance, while unexpected worker faults remain pending with retry metadata and an operator-visible error; and

- repeated reads of one EPC remain one physical item, not increasing inventory.

The batch status endpoint reports accepted, processed, rejected, and pending counts, including retries linked to an existing event.

Stable Zone and Confidence

Recent per-EPC evidence is kept in event-worker memory for the hosted slice. The defaults are three consistent reads in ten seconds, a median-RSSI tie-break between competing zones, and retaining the previous zone at reduced confidence when evidence is ambiguous. Confidence deterministically considers read count, RSSI separation, recency, and reader health.

These thresholds must be calibrated in an Orange pilot. Process-local evidence keeps the demo small but means a worker restart rebuilds the candidate window from new reads. Durable event identity, the current-state version, and conditional database updates still prevent duplicates or older evidence from corrupting confirmed state.

The design allows removal after 30 minutes without evidence only while connectivity is healthy. A store is **LIVE**, **DEGRADED**, or **STALE** based on heartbeat age, live-event age, buffered-event backlog, and reader coverage. After reconnection it remains stale until buffered observations drain and live reads resume. Stale or low-confidence inventory cannot create automatic replenishment tasks.

State, Projection, and Replay Safety

For a confirmed transition the worker conditionally updates **current_item_state**, increments **state_version**, and inserts a deterministic transition outbox row in the same transaction. The transition ID is derived from **tenant_id + epc + state_version**.

The event worker applies signed deltas to **inventory_projection**. A deterministic delta ID deduplicates each bucket update. A backroom-to-floor move therefore produces **-1** in the old zone and **+1** in the new zone exactly once at the projection boundary. The projection is eventually consistent and can be rebuilt from **current_item_state** with the reconciliation CLI.

Why PostgreSQL Instead of Kafka in the Demo

PostgreSQL makes **202 Accepted** and durable work one atomic commit and avoids an extra service that can fail during reviewer testing. The downside is lower streaming throughput and a polling worker. I would first use edge coalescing, larger batches, bounded worker concurrency, connection pooling, and time/tenant partitioning.

When measured throughput, long replay windows, or multiple independent consumers justify it, the inbox publisher can feed Kafka-compatible topics. Raw observations would be keyed by **tenant_id:epc**; inventory deltas by **tenant_id:store_id:sku_id:zone_id**. S3 would hold long-retention raw evidence. These are production extensions, not hidden runtime requirements.

Implemented APIs

- **POST /v1/rfid/observation-batches**
- **GET /v1/rfid/observation-batches/{batch_id}**
- **GET /v1/stores/{store_id}/inventory**
- **GET /v1/items/{epc}**

4. Identity, Access, and User Management

Authentication and authorization are separate. The hosted demo uses Argon2id password hashes and short-lived JWTs. A token contains user and tenant identity; every request reloads current roles and store assignments from PostgreSQL. Revoked access therefore does not remain valid until token expiry.

The canonical roles are:

- **TENANT_ADMIN**: tenant setup, users, catalog, policy, inventory, and replenishment administration;
- **CORPORATE_USER**: read access across every store in the tenant;
- **STORE_MANAGER**: management and execution within explicitly assigned stores; and
- **STORE_ASSOCIATE**: inventory visibility and task execution within explicitly assigned stores.

Application authorization checks store scope before querying. Forced RLS independently prevents cross-tenant rows from being visible. Tests prove Orange cannot read another tenant, a Store 1 user cannot read Store 2 without assignment, and a corporate user can read all stores inside Orange.

JWT is appropriate for a reproducible reviewer login, but it is not an enterprise identity strategy. Production would replace password authentication with Orange's OIDC/SAML provider and optional SCIM provisioning while keeping the same database-backed resource authorization.

Implemented APIs

- **POST /v1/auth/login** (demo helper)
- **GET /v1/me**
- **POST /v1/users**
- **PUT /v1/users/{user_id}/roles**
- **PUT /v1/users/{user_id}/store-assignments**
- **GET /v1/users** and **GET /v1/users/{user_id}**
- **POST /v1/users/{user_id}:suspend**
- **GET /v1/users/audit-records**

5. Replenishment Policy and Execution

Versioned Policies

A policy has immutable activated versions and editable drafts. Activation retires the previous active version. Rules support tenant default, category, style, and SKU/size selectors with optional store scope.

Resolution is deterministic, highest first:

1. Store + SKU/size
2. Store + style
3. Store + category
4. Tenant + SKU/size
5. Tenant + style
6. Tenant + category
7. Tenant default

Explicit priority resolves equal specificity. Equal-specificity, equal-priority overlap is rejected rather than resolved arbitrarily. Evaluation stays at SKU/size level, so a style total cannot hide a missing apparel size.

Calculation and Safety Gates

```
if floor_qty >= min_floor_qty:
    required_qty = 0
else:
    required_qty = min(
        backroom_qty,
        max(0, target_floor_qty - floor_qty - open_task_qty)
    )
```

A task is created only if the result is positive, the store is live, confidence meets the configured threshold, and no active task already covers the SKU shortage. A PostgreSQL partial unique index permits only one **OPEN**, **CLAIMED**, or **IN_PROGRESS** task per tenant/store/SKU, so concurrent evaluations fail safely instead of duplicating work.

The lifecycle is **OPEN** -> **CLAIMED** -> **IN_PROGRESS** -> **COMPLETED**, with manager-controlled **CANCELED** and **EXPIRED** terminal alternatives. Optimistic task versions protect concurrent updates.

The demo uses an explicit evaluation endpoint. In production, inventory transitions can enqueue targeted store/SKU evaluation and policy activation can fan out in bounded batches. Keeping that fan-out outside the take-home avoids pretending a large estate scheduler was tested.

Implemented APIs

- **POST /v1/replenishment-policies**
- **POST /v1/replenishment-policies/{policy_id}/versions**
- **PATCH /v1/replenishment-policy-versions/{version_id}**
- **POST /v1/replenishment-policy-versions/{version_id}/activate**
- **POST /v1/replenishment/evaluations**
- **GET /v1/stores/{store_id}/replenishment-tasks**
- **PATCH /v1/replenishment-tasks/{task_id}**

Reviewer Contract and Delivery

GET /openapi.json is the authoritative OpenAPI 3.1 contract. Swagger is at **/docs**; the application release is **0.2.0**. Earlier prototype routes remain only as migration-test fixtures and are not mounted by the submitted application.

Authentication Surfaces

Surface	Credential	Purpose
Platform setup	X-Platform-Key	Tenant, store, and catalog setup
RFID gateway	X-Device-Token	Observation-batch submission only
Orange user	Bearer JWT	User, inventory, policy, evaluation, and task APIs

Required Commands

```
docker compose up --build
make migrate
make seed
make test
make demo
```

Compose starts PostgreSQL, migrations, the API, the catalog worker, and the event worker. **make demo** runs the canonical vertical slice, including duplicate/late-event and store-scope checks. The RFID simulator additionally generates normal reads, exact retries, repeated reads, late/out-of-order events, adjacent-zone conflicts, unknown EPCs, outage replay, and stationary bursts.

Hosting

render.yaml defines a free-demo web service whose supervised launcher runs the API and both worker entry points. Compose keeps them as separate processes. It intentionally expects an externally provisioned managed PostgreSQL database rather than coupling the code to one database vendor.

Deployment needs:

1. a PostgreSQL owner URL for Alembic;
2. a separate non-owner **abacus_app** runtime URL;
3. a **PLATFORM_API_KEY** secret (**JWT_SECRET** is generated by the Blueprint);
4. a reviewer admin password supplied through the host secret store; and
5. repository access for the hosting provider and reviewer.

The application role must exist before the first migration so Alembic can grant table, sequence, function, and RLS access. A free Render web instance sleeps when idle, including its workers; the launcher also retains the owner URL because pre-deploy commands are paid-only. Those are explicit demo compromises. A production deployment uses separate always-on workers and keeps the owner credential out of the runtime container. Automatic deployment should remain off after the final tagged commit. The final submission should include the base URL, **/docs**, **/openapi.json**, **/health/ready**, **/version**, and reviewer credentials sent outside the repository.

Scale Path: 100 to 5,000 Stores

Store count alone does not size RFID traffic. The important inputs are readers and antennas per store, post-edge-filter observations per second, burst/offline replay volume, EPC cardinality, retention, consumer count, and freshness SLO.

For the initial estate I would retain edge aggregation and buffering, batch ingestion, one vertically scaled RFID inference worker, connection pooling, partitioned history, and archive retention policies. Catalog workers can scale with safe claiming. RFID workers require EPC partition affinity or shared evidence state before replicas are safe; production Kafka partitioning by **tenant_id:epc** supplies that boundary.

For sustained multi-consumer load or long replay windows I would evolve to regional cells:

```
Readers -> edge gateway -> regional ingress -> Kafka-compatible stream -> EPC processors
                                     |                               |
                                     +-----> S3 raw archive         v
                                               regional PostgreSQL
```

Kafka adds partitioned ordering, replay, and independent consumers, but also broker operations, partition planning, monitoring, and cost. It should be introduced for measured needs, not to make a small take-home look distributed.

Key Tradeoffs and Honest Boundaries

Selected approach	Alternative	Decision
Shared schema + forced RLS	Database per tenant	Efficient onboarding with database-enforced isolation; dedicated cells remain possible
PostgreSQL durable inbox	Kafka/SQS from day one	Atomic acceptance and fewer reviewer dependencies; broker is the measured-scale path
PostgreSQL catalog staging	MinIO/S3 in the demo	Atomic promotion with one dependency; object storage is the large-file/retention path
Effective-dated assignments	Mutable device location	Preserves late-event meaning; a GiST exclusion constraint prevents overlapping history
In-memory recent RFID evidence	Distributed stateful stream processor	Small and inspectable; a restart rebuilds the evidence window from new reads
Derived inventory projection	Direct counter writes from each read	Deduplicated, auditable transitions and rebuildability at the cost of eventual consistency
Versioned deterministic policies	General rules engine	Easier validation, precedence, explanation, and testing
Demo JWT	Enterprise SSO in the demo	Reproducible review; production authentication moves to OIDC/SAML/SCIM

Implementation boundaries:

- Kafka/Redpanda, SQS, MinIO/S3, Kubernetes, Flink, and regional cells are not deployed.
- Processing is at least once with idempotent effects, not “exactly once.”
- Recent stable-zone evidence is process-local in the hosted scope.
- No throughput claim is made for 5,000 stores without an agreed workload and load test.
- RFID presence is observed inventory, not a replacement for POS, receiving, transfer, return, or shrink ledgers.
- Production confidence thresholds and replenishment business rules require an Orange pilot.
- Physical commissioning, real reader integration, enterprise identity, and book-inventory variance remain production work.

Verification Evidence

The repository includes PostgreSQL migrations, forced RLS policies, a non-superuser runtime role, unit and integration tests, seed data, sample CSV/EPC data, an RFID simulator, a smoke test, a full demo script, Docker Compose, a container build, CI checks, Swagger/OpenAPI, and an architecture diagram.

Release gates are Ruff, mypy, the full PostgreSQL test suite, Alembic upgrade/drift, the image build, Compose validation, and the end-to-end demo. Add the hosted URL and release tag only after the deployed commit passes them.